MICROBIT. УРОК №1. ДИАГНОСТИКА СИСТЕМ.

Всем привет! Добро пожаловать в лабораторию разработки игр! В ближайшие недели мы с вами пройдем путь, который в 80-х годах совершали инженеры Nintendo, Sega и Atari. Мы не просто будем играть, мы создадим собственное устройство, на котором эти игры работают.

Сегодня — самый важный день. Представьте, что вы работаете в секретном отделе разработки новой консоли (назовем её Bit-Boy). Конвейер только что выпустил первую партию прототипов. Прежде чем писать Марио или Танчики, инженер обязан задать главный вопрос: **А железо вообще работает?**

Наша задача сегодня — научить мозг — **Micro:bit** — понимать команды от джойстика и кнопок.

Представьте, что вы купили новый принтер и подключили его к компьютеру. Будет ли он печатать сразу? Скорее всего, нет. Компьютеру нужен **Драйвер**.

> ## Драйвер
> Это специальная программа-переводчик. Она объясняет процессору, как именно нужно общаться с этим конкретным устройством.

Сегодня мы поступим как профессиональные программисты. Мы не будем писать сложный код управления электричеством в каждой игре. Мы подключим свой **драйвер** один раз, а потом будем пользоваться простыми командами типа **нажата_кнопка_A()**.

Итак, перед вами лежит комплект деталей. Давайте проведем анбоксинг и посмотрим, из чего состоит наша будущая консоль.

> ## Интересный факт!
>
> Анбоксинг (от англ. Unboxing - распаковка) — это не просто популярный жанр на YouTube, где блогеры открывают новые гаджеты. В профессиональной инженерии этот этап называется Входной контроль (от англ. Incoming Inspection). На заводах Apple или Sony инженеры вскрывают коробки с деталями не ради лайков, а чтобы проверить качество. Найти брак на этапе распаковки стоит 1 доллар, а найти его, когда устройство уже собрано и продано — 1000 долларов! Сегодня мы с вами проводим именно такую инженерную инспекцию.

Мозг вашей будущей консоли: микрокомпьютер **BBC Micro:Bit**, продемонстрированный на *рисунке 1*.

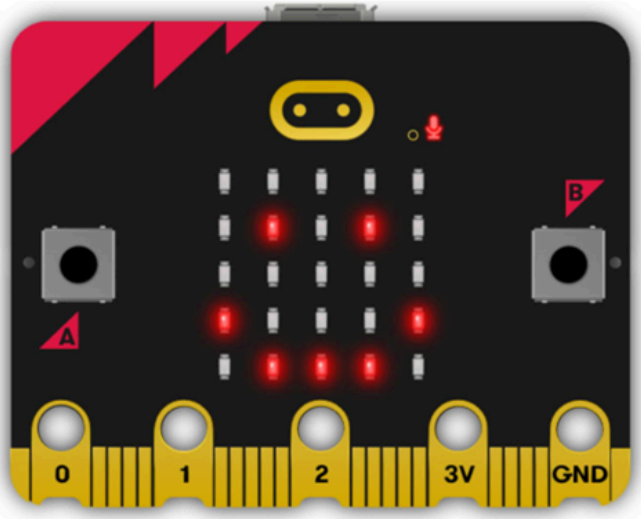


Рисунок 1

Это сердце нашей системы.

- **Процессор:** Nordic nRF52833 (гораздо мощнее, чем компьютеры, отправлявшие людей на Луну).
- **Матрица 5×5:** Наш экран. 25 красных светодиодов.
- **Edge Connector (Золотая гребенка снизу):** Это нервные окончания. Через них Micro:bit общается с внешним миром.

Тело вашего устройства — Yahboom Game Handle, продемонстрированный на *рисунке 2*.



Рисунок 2

Сам по себе Micro:bit неудобен для мини-игр — у него всего две кнопки. Поэтому мы используем **Плату расширения (от англ. Shield)** с дополнительными компонентами для управления.

> ***Справка: Зачем нужна Плата расширения (Shield)?***
>
> *Micro:bit — это мозг. Но мозгу нужны удобные руки и ноги.*
>
> *Как мы уже отметили, у Micro:bit снизу есть золотистая гребенка с контактами. Подключать к ним провода напрямую неудобно — они будут выпадать, а держать плату в руках во время игры невозможно.*
>
> ***Плата расширения (от англ. Expansion Board)** — это тело для нашего мозга. В инженерном мире такие платы часто называют **Шилдами (от англ. Shield — Щит)**.*
>
> *Она выполняет три задачи:*
>
> - ***Коммутация**: берет тонкие контакты Micro:bit и надежно соединяет их с кнопками.*
> - ***Эргономика**: превращает голую плату в удобный геймпад, который приятно держать в руках.*
> - ***Питание**: на ней есть батарейный отсек, чтобы консоль работала без проводов.*

Посмотрите на представленную плату внимательно:

- **Джойстик:** позволяет управлять движением (лево, право, вверх, вниз).
- **4 цветные кнопки:** Синяя (A), Желтая (B), Красная (X), Зеленая (Y).

- **Вибромотор:** для тактильной отдачи (как в геймпадах других консолей).

> ***Техническая справка: Переключатель вибрации***
>
> *Найдите на плате маленький переключатель с надписью **Vibration Switch**.*
>
> - *ON**:** Джойстик вибрирует сам по себе при нажатии.*
> - *OFF: Вибрация включается только по команде из кода.*
>
> *Переведите переключатель в положение OFF. Мы хотим полного контроля!*

Прежде чем оживить устройство, нам нужен инструмент управления. Как вы уже знаете, в IT мире такой инструмент называют **IDE (Integrated Development Environment) — Интегрированная Среда Разработки**.

Мы будем использовать **официальный онлайн-редактор: micro:bit Python Editor** *(рис. 3)*

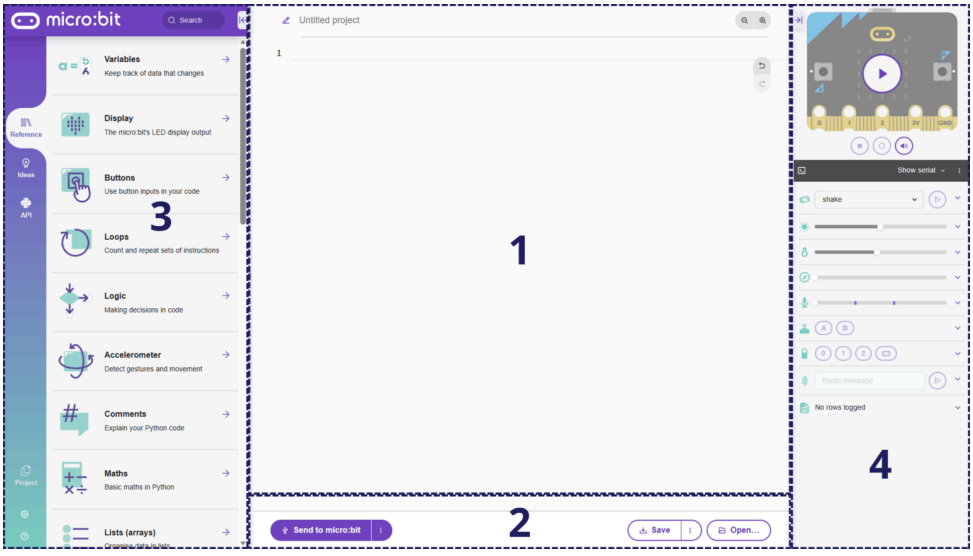


Рисунок 3

Посмотрите на экран. Наш пульт управления состоит из трех частей:

1. Редактор кода (Центр): здесь мы пишем команды на уже знакомом вам из курса Lego Pro языке **MicroPython**. Это своего рода переговорная комната, где мы объясняем процессору, что делать.
2. Консоль и Загрузка (Снизу): Кнопка **Send to micro:bit** отправляет наш код по проводу прямо в микрокомпьютер
3. Библиотека (Слева) с примерами кода для различных встроенных и подключаемых компонентов
4. Симулятор (Справа): Виртуальный Micro:bit. Если у вас нет устройства под рукой, код можно проверить прямо тут. Он покажет смайлики и даже нажмет виртуальные кнопки, однако не сможет сымитировать компоненты геймпада.

Этап Подключение

Чтобы браузер увидел вашу сборку, нужно их «познакомить». Подключите Micro:bit кабелем к USB. Нажмите кнопку с тремя точками рядом с «Send to micro:bit».

Выберите **Connect** *(рис. 4)*.

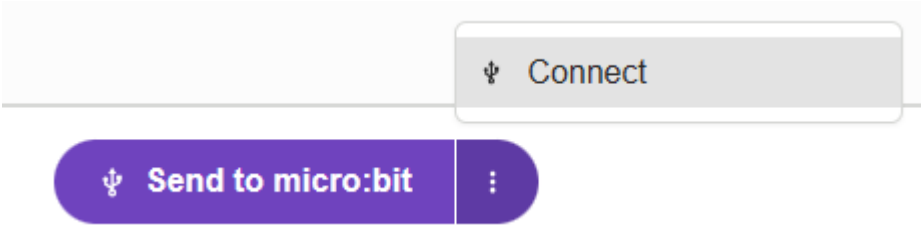


Рисунок 4

В появившемся окне выберите ваше устройство и нажмите кнопку **Подключиться** *(рис. 5).*

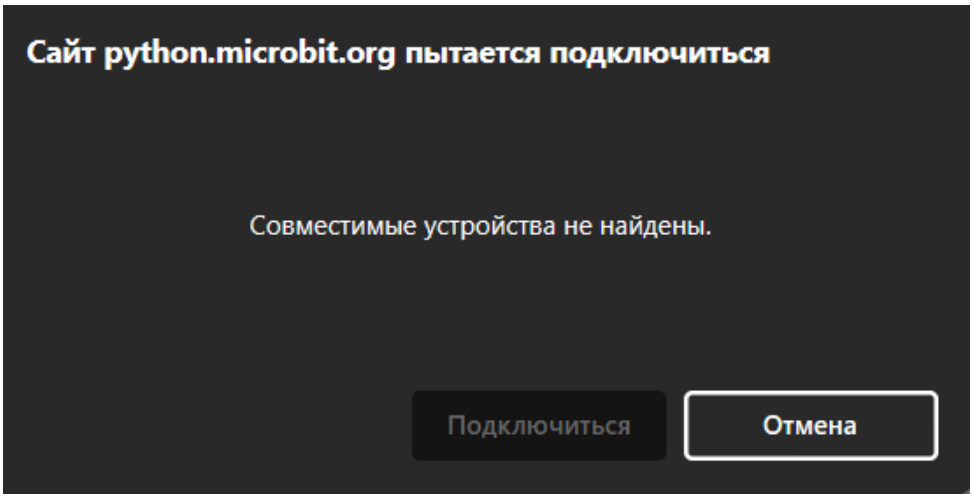


Рисунок 5

Если иконка стала зеленой — поздравляем, канал связи установлен!

Практическая работа №1: Установка Драйвера

В среде разработки MicroPython мы можем создавать дополнительные файлы. Наш драйвер будет лежать в отдельном файле **gamepad.py.**

Откройте среду разработки. Слева от кнопки **Send to micro:bit** найдите кнопку **Project** (иконка папки/файлов). Нажмите на неё. Перед вами откроется папка вашего проекта. Здесь мы можете увидеть главный файл вашего проекта – **main.py** *(рис. 6).*

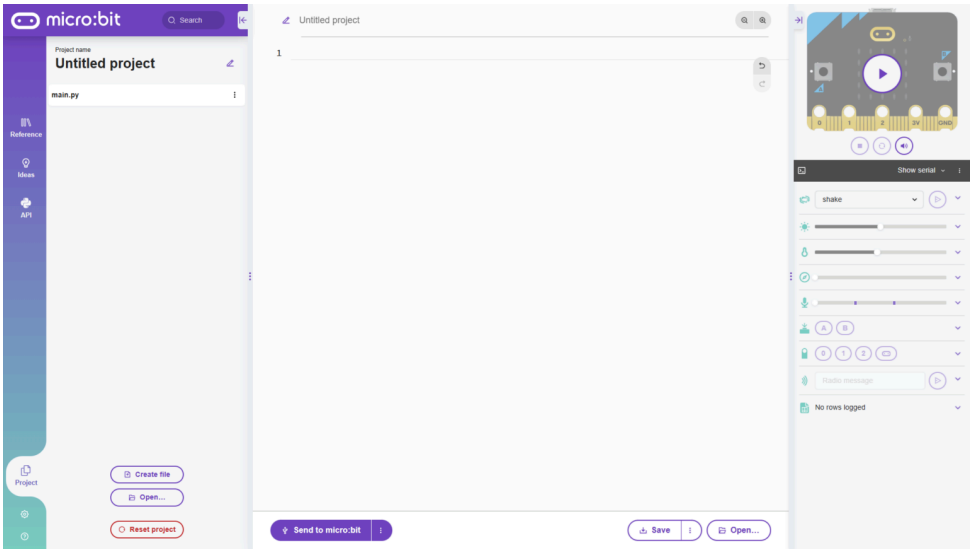


Рисунок 6

В **[дополнительных материалах]** к уроку приложен указанный ранее файл — **gamepad.py.**

Вы можете загрузить его с вашего ПК с помощью кнопки **Open...** Либо создать файл новый файл с именем **gamepad.py (обязательно с маленькой буквы!),** нажав кнопку **Create file**.

В этом случае скопируйте код драйвера и вставьте в среду разработки *(рис. 7)*.

Справка!

Код драйвера делает сложную работу: он настраивает пины и электрические сигналы, чтобы мы могли не думать об этом.

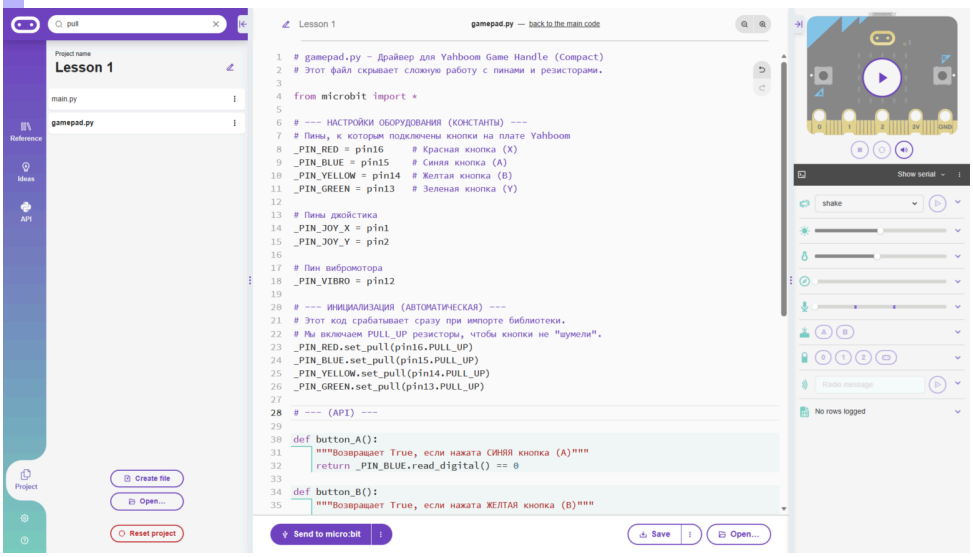


Рисунок 7

Практическая работа №2: Тест кнопок

Теперь переключаемся обратно в главный файл **main.py**. Нам нужно подключить наш новый драйвер. Для этого используется команда **import**.

Напишем программу, которая проверяет работу кнопок. В первую очередь подключим библиотеки:

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер
```

Рассмотрим код подробнее:

- Первая строка подключает все основные возможности micro:bit
- Вторая строка подключает в основной файл ваш файл-драйвер

Далее по традиции запустим приветствие:

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер

display.scroll("Hello Bit!")
```

Рассмотрим подробнее новую строку:

- *display.scroll(«Hello Bit!»)* – **display** – основной модуль для управления матрицей. Метод **scroll()** позволяет прокрутить строку, указанную в скобках, побуквенно слева-направо. Помните! Текстовые данные пишутся всегда в кавычках!

Далее добавим проверку первой (красной) кнопки – X.

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер

display.scroll("Hello Bit!")

while True:
    # Проверяем кнопку (X)
```

```
    if gamepad.button_X():
        display.show("X")
```

Рассмотрим код подробнее:

- Внутри бесконечного цикла **while True:**
- Запускаем условную конструкцию **if**, с проверкой функции кнопки X из вашего драйвера.
- В случае успеха – на экран будет выведен, благодаря методу **show()**, символ **X**

Теперь загрузите код в Micro:bit и нажмите Красную кнопку — должна появиться буква X.

Самостоятельная работа!

Допишите программу так, чтобы она распознавала оставшиеся три кнопки: Синюю (A), Желтую (B) и Зеленую (Y). Пусть они показывают соответствующие буквы на экране.

Ответ:

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер

display.scroll("Hello Bit!")

while True:
    if gamepad.button_A():
        display.show("A")

    elif gamepad.button_B():
        display.show("B")

    elif gamepad.button_Y():
        display.show("Y")

    elif gamepad.button_X():
        display.show("X")
```

> ***Обратите внимание!*** *В данном случае используется часть условной конструкции* ***elif*** *– альтернативное условие. Таким образом будет гарантировано отсутствие прерываний программы – условная конструкция проверяет все условия последовательно.*

Практическая работа №3: Джойстик и Аналоговые сигналы

Кнопки работают просто: нажата или нет (0 или 1). Джойстик устроен сложнее. Он выдает число от **0 до 1023**.

- 0 — это крайнее положение (например, слева).
- 1023 — это другое крайнее положение (справа).
- ~512 — это центр.

Давайте посмотрим на эти числа.

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер

while True:
    # Получаем координаты стика
    x = gamepad.joystick_x()
    y = gamepad.joystick_y()

    # Выводим их в консоль компьютера
    print("X:", x, "Y:", y)

    sleep(100)
```

Проведем эксперимент:

1. Загрузите код.

2. Нажмите кнопку **Show Serial** под редактором кода *(рис. 8)*.

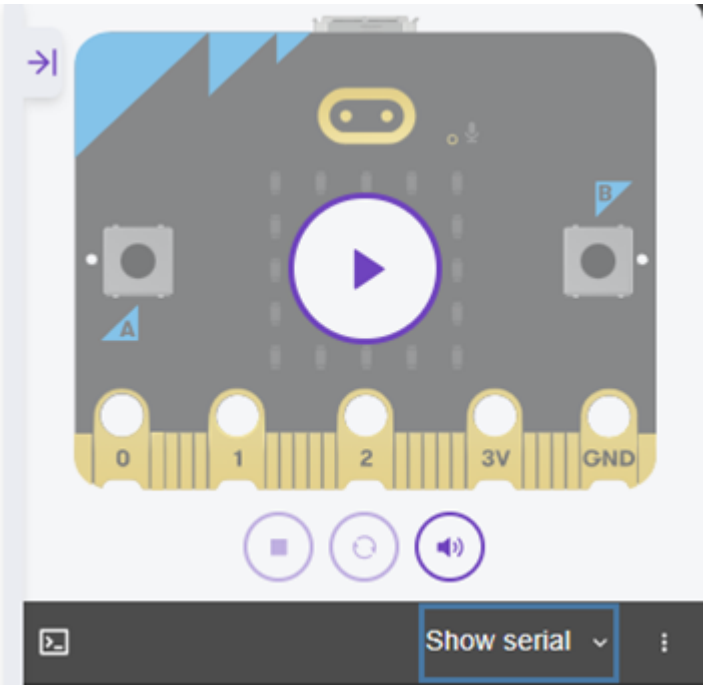


Рисунок 8

3. Покрутите стиком. Вы увидите бегущие графики.

4. Запишите, где у вашего джойстика Верх, а где Низ (числа могут отличаться!).

Практическая работа №4: Визуализация направления

Превратим сухие цифры в стрелки на экране с помощью уже знакомой вам условной конструкции:

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер

while True:
    x = gamepad.joystick_x()

    # Если стик отклонен сильно влево (число меньше 200)
    if x < 200:
        display.show(Image.ARROW_W) # West (Запад)
```

```
    # Если стик отклонен сильно вправо (число больше 800)
    elif x > 800:
        display.show(Image.ARROW_E) # East (Восток)

    else:
        display.clear()
```

В зависимости от того, в какую сторону будет отклонен стик джойстика, ваш микрокомпьютер отобразит вам соответствующую стрелку на матрице.

Самостоятельная работа!

Дополните программу выводом стрелок вверх и вниз при отклонении стика по оси Y.

Ответ:

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер

while True:
    x = gamepad.joystick_x()
    y = gamepad.joystick_y()

    if x < 200:
        display.show(Image.ARROW_W)
    elif x > 800:
        display.show(Image.ARROW_E)
    elif y < 200:
        # ВНИМАНИЕ: Проверьте свой джойстик.
        # Часто 0 - это Верх (North), но бывает наоборот!
        display.show(Image.ARROW_N)
    elif y > 800:
        display.show(Image.ARROW_S)
    else:
        display.clear()
```

Практическая самостоятельная работа №5: System Check Utility

Теперь объединим всё, чему научились. Мы создадим профессиональную утилиту для проверки геймпада.

Техническое задание:

1. При нажатии кнопок показывать их первую букву.
2. **!!!** Если кнопки не нажаты — показывать направление джойстика стрелками.

Ответ:

```
from microbit import *
import gamepad  # Подключаем наш файл-драйвер

display.show(Image.HAPPY)
sleep(500)

while True:
    # 1. Сначала проверяем кнопки
    if gamepad.button_A():
        display.show("A")

    elif gamepad.button_B():
        display.show("B")

    elif gamepad.button_Y():
        display.show("Y")

    elif gamepad.button_X():
        display.show("X")
        gamepad.vibrate(100) # Короткая вибрация!
        sleep(200) # Пауза, чтобы не вибрировал сплошным

    # 2. Если кнопки не трогали, проверяем джойстик
    else:
        x = gamepad.joystick_x()
        y = gamepad.joystick_y()

        if x < 200:
            display.show(Image.ARROW_W)
```

```
elif x > 800:
    display.show(Image.ARROW_E)
elif y < 200:
    # ВНИМАНИЕ: Проверьте свой джойстик.
    # Часто 0 - это Верх (North), но бывает наобо
    display.show(Image.ARROW_N)
elif y > 800:
    display.show(Image.ARROW_S)
else:
    display.clear()
```

Итак, мы с вами проделали огромную работу. Мы не просто написали код, мы **подключили библиотеку**, протестировали аналоговые и цифровые датчики!
Резюме инженера:

- **Драйвер (gamepad.py)** — это файл, который прячет сложность и дает нам удобные команды.
- **Цифровой сигнал** — это кнопки (Да/Нет).
- **Аналоговый сигнал** — это джойстик (Диапазон 0..1023).

На следующем уроке этот файл **gamepad.py** станет основой для нашей первой игры — космического

шутера. Сохраните его!